Machine Learning for Network Intrusion Detection

CIC-IDS2017 Dataset | Final Project Report

Joselyn Zamora — Miranda Macias — Deniss Garcia

May 2026

Introduction

Cybersecurity threats are constantly evolving, and automated detection systems are critical for preventing data breaches and system damage. Traditional rule-based intrusion detection systems struggle to keep pace with the volume and variety of modern attacks. Machine learning offers a promising alternative: models that can learn attack patterns from labeled network traffic data and generalize to new threats without manual rule updates.

This project builds and evaluates machine learning models for network intrusion detection using the publicly available CIC-IDS2017 dataset from the Canadian Institute for Cybersecurity. We downloaded the raw data from official sources and performed our own preprocessing and feature engineering. Our goal was to train and compare 3–4 machine learning models, handle class imbalance, and evaluate performance using precision, recall, and F1-score — metrics that reflect real-world detection needs where missing an attack (low recall) is costly.

We structured our evaluation across three stages of increasing realism. Stage 1 establishes a random-split baseline. Stage 2 introduces a temporal split, training on Tuesday traffic and testing on Wednesday traffic, to simulate real deployment where models must handle new attack types they have never seen. Stage 3 applies active learning to explore whether selective labeling can recover performance after distribution shift. The four models evaluated are Decision Tree, Logistic Regression, k-Nearest Neighbors, and a Multilayer Perceptron (Neural Network).

Data Cleaning and Preprocessing

Overview

The raw CIC-IDS2017 dataset consists of 8 CSV files totaling 2,830,743 rows of network flow records. Before any modeling could take place, we performed a multi-step cleaning pipeline to produce a reliable, analysis-ready dataset. The process was implemented in two complementary scripts: a per-file cleaner (individual_cleanup_data.py) that processes each CSV independently, and a master notebook (data_cleaning.ipynb) that merges, cleans, and finalizes the combined dataset.

Step 1: Loading the Raw Data

We loaded all 8 CSV files from the data/ directory using pandas with low_memory=False to suppress mixed-type warnings, then concatenated them into a single master DataFrame. This yielded 2,830,743 rows before any cleaning was applied.

Step 2: Column and String Normalization

The raw CSV files contain leading and trailing whitespace in column names, which causes silent key-lookup failures in downstream code. We stripped all column names using str.strip() and also removed any columns whose names began with “Unnamed”, which are artifact columns introduced by index leakage during export. For string-typed feature columns, we applied the same strip operation to cell values to ensure consistent matching.

Step 3: Handling Infinity and Missing Values

Several computed flow-rate features (Flow Bytes/s and Flow Packets/s) produce infinite values when flow duration is zero. We replaced all occurrences of positive and negative infinity — including string variants such as “Infinity” and “inf” — with NaN so that pandas could treat them uniformly as missing. The affected columns and their missing-value counts before removal were:

Flow Bytes/s: 2,867 missing — Flow Packets/s: 2,867 missing — Active Max / Idle Max: up to 2,544,276 missing (duplicated columns from file merging)

We applied a two-pass filtering strategy. First, any column missing more than 50% of its values was dropped entirely (axis=1). This removed the heavily-incomplete Active Max and Idle Max duplicate columns. Second, any remaining row containing at least one NaN was dropped (axis=0), removing the small number of rows affected by the flow-rate infinities.

Step 4: Label Binarization

The original Label column contains multi-class text labels (e.g., BENIGN, FTP-Patator, DoS Hulk). We converted these to a binary Attack_Flag: 0 for BENIGN traffic and 1 for any attack class. This aligns with the binary detection framing of the project. The original Label column was then dropped.

Outcome

After all cleaning steps, the master dataset was reduced from 2,830,743 rows to 2,096,135 rows, with 1,712,215 normal flows (Attack_Flag = 0) and 383,920 attack flows (Attack_Flag = 1), yielding a class imbalance ratio of approximately 4.46:1. The final cleaned dataset was saved as CIC_IDS2017_Clean.csv and used as the input for all downstream modeling experiments.

Classical Model Comparison for Network Intrusion Detection

Overview

This writeup covers three experimental stages on the CIC-IDS2017 intrusion detection benchmark:

1. **Stage 1 — Random-split baseline**: standard 80/20 stratified split, used to establish per-model performance and address the class-imbalance commitment in our proposal.
2. **Stage 2 — Temporal split (Tuesday → Wednesday)**: train on Tuesday traffic, test on Wednesday traffic. A realistic deployment setup that exposes how poorly the random-split models generalize to attack types they haven't seen.
3. **Stage 3 — Active learning**: use entropy-based uncertainty sampling to selectively label a small number of Wednesday samples and retrain. Includes a random-sampling control, which revealed an unexpected and instructive failure mode.

Stage 1: Random-Split Baseline

Approach

We trained three classical classifiers on the cleaned CIC-IDS2017 dataset (~2.1M rows, 78 features, binary Attack_Flag target): Decision Tree, Logistic Regression, and k-Nearest Neighbors. We selected these three because they cover meaningfully different inductive biases — a threshold-based splitter, a linear boundary, and a non-parametric distance-based learner — which makes the comparison more informative than picking three variants of the same idea.

Per the project proposal, we also addressed class imbalance by training balanced versions of the Decision Tree and Logistic Regression using scikit-learn's class_weight="balanced" setting. kNN doesn't natively support class weighting, so it was run once.

Methodology

We used a stratified 80/20 train/test split (random_state=42). The dataset has a roughly 4.46:1 imbalance (1,712,215 normal flows to 383,920 attack flows). Features were standardized with StandardScaler for Logistic Regression and kNN, both of which are scale-sensitive. The Decision Tree was trained on the unscaled features since its splits don't depend on feature magnitude. kNN training was capped at 100,000 rows to keep predict-time tractable; evaluation is on the full test set for all models.

Hyperparameters: Decision Tree (max_depth=20), Logistic Regression (max_iter=1000), kNN (k=5).

Results

Model	Precision	Recall	F1	Predict time
Decision Tree	0.9984	0.9987	**0.9985**	0.13s
Decision Tree (balanced)	0.9978	0.9991	0.9985	0.10s
kNN (k=5)	0.9913	0.9913	0.9913	93.46s
Logistic Regression	0.9404	0.9457	0.9430	0.06s
Logistic Regression (balanced)	0.7872	0.9692	0.8687	0.05s

Confusion matrices (rows = true class, cols = predicted class; class 0 = normal, class 1 = attack):

```
Decision Tree            Decision Tree (balanced)    kNN (k=5)
[[342318    125]         [[342277    166]          [[341775    668]
 [   103  76681]]         [    66  76718]]           [   666  76118]]

Logistic Regression        Logistic Regression (balanced)
[[337839   4604]          [[322321  20122]
 [  4173  72611]]          [  2367  74417]]
```

Discussion

The Decision Tree came out on top. Network flow features (packet sizes, flow durations, flag counts, byte rates) split cleanly on numeric thresholds, which is exactly what tree learners exploit. The unweighted DT missed only 103 attacks out of 76,784 in the test set.

Logistic Regression lagged behind at 0.943 F1, suggesting the boundary between benign and attack flows isn't well-approximated by a hyperplane. kNN sat between the two at 0.991 F1 but came with a ~93-second predict time on a capped training set — not deployable for real-time intrusion detection.

Class weighting barely moved the Decision Tree (F1 stayed at 0.9985) since the unweighted model was already fitting both classes well. For Logistic Regression, weighting traded precision for recall: recall up from 0.9457 to 0.9692, precision down from 0.9404 to 0.7872. By F1 the unweighted version wins, but a high-security environment might prefer the balanced version for its higher attack catch rate.

Stage 2: Temporal Split (Tuesday → Wednesday)

Motivation

Stage 1's near-perfect F1 reflects an easy setup: random sampling guarantees training and test sets come from the same distribution. Real deployment isn't like that. New attack types appear over time, and a model trained on yesterday's data has to handle today's traffic having never seen it. To assess this, we split temporally: train on Tuesday, test on Wednesday.

The choice of days matters. Monday in CIC-IDS2017 contains zero attacks (purely benign traffic), so it can't serve as a training set for a binary classifier. We chose Tuesday → Wednesday because:

- Tuesday contains brute-force attacks (FTP-Patator, SSH-Patator) — many small repetitive login attempts.
- Wednesday contains DoS attacks (Hulk, GoldenEye, Slowloris, Slowhttptest) plus Heartbleed — high-volume timing-based patterns with completely different feature signatures.
- Both days contain attacks, so the classifier has something to learn and something different to be tested on.

This is the cleanest possible distribution-shift experiment with this dataset.

Methodology

Same three model families as Stage 1 (DT, LR, kNN), each with class_weight="balanced". Same preprocessing. Hyperparameters held constant across stages to isolate the effect of the split.

- Tuesday (training): 445,645 rows — 431,813 benign + 13,832 attacks (3.1% attack rate)
- Wednesday (test): 691,406 rows — 439,683 benign + 251,723 attacks (36.4% attack rate)

Results

Model	Attack Precision	Attack Recall	Attack F1	Accuracy
Decision Tree (balanced)	0.9450	0.0008	0.0015	0.6362
Logistic Regression (balanced)	0.5663	0.0403	0.0752	0.6394
kNN (k=5)	0.8731	0.0020	0.0041	0.6366

Discussion

Attack recall collapsed from ~99.9% in Stage 1 to below 5% across all three models. The Decision Tree, our best Stage 1 performer, dropped to 0.08% recall — it caught 188 out of 251,723 Wednesday attacks.

The model trained on Tuesday's brute-force patterns simply cannot recognize Wednesday's DoS patterns; the feature signatures are too different.

What's interesting is that precision held up. The DT's precision on its 199 attack predictions was 94.5% — when it does flag something, it's usually right. It's just only willing to flag samples that look like Tuesday's brute force traffic, and Wednesday's DoS doesn't qualify. Logistic Regression survived marginally better than the others (0.04 recall vs 0.001-0.002) because its smoother linear boundary catches a small fraction of DoS samples by chance, while the Decision Tree's hard thresholds fail almost completely.

This result is the entire motivation for the active learning experiment that follows. A model trained on yesterday's attack types is essentially blind to new attack types — but labeling all of today's traffic is infeasible. Can selective labeling close the gap?

Stage 3: Active Learning with Entropy-Based Sampling

Methodology

We used Logistic Regression as the AL model — calibrated probability outputs and fast retraining make the loop practical. The procedure:

1. Train LR on Tuesday data.
2. Predict probabilities on all 691k Wednesday samples.
3. For each sample, compute binary entropy: $H = -p \cdot \log(p) - (1-p) \cdot \log(1-p)$, peaking at $p=0.5$.
4. Select the top 500 highest-entropy samples and reveal their true labels.
5. Add them to training, retrain.
6. Evaluate on the remaining 690,906 Wednesday samples.

To check whether any improvement comes from smart selection or just more data, we added a random-sampling control: same procedure with 500 randomly chosen samples instead. If entropy-based AL is doing real work, it should outperform random selection.

Results

Scenario	Attack Precision	Attack Recall	Attack F1	Accuracy	Attacks/500
(a) Baseline: Tuesday → Wednesday	0.5663	0.0403	0.0752	0.6394	—
(b) Entropy AL (+500 uncertain)	0.6177	0.0410	0.0769	0.6414	**0 of 500**
(c) Random control (+500 random)	**0.9177**	**0.6452**	**0.7577**	0.8498	**184 of 500**

Discussion

The control vastly outperformed the active learning method — a surprising result that turns out to be highly informative.

The mechanism is visible in the last column. Of the 500 entropy-picked Wednesday samples, zero were attacks. The model didn't pick a single DoS example. Random sampling, by contrast, naturally caught 184 attacks (approximately 36% of 500, matching Wednesday's natural attack rate).

This exposes a well-known failure mode of pure uncertainty sampling: it surfaces samples where the model's predicted probability is near 0.5, which are borderline benign samples. The model is confidently wrong about Wednesday's DoS attacks — it classifies them as benign with high confidence because they look nothing like Tuesday's brute-force attacks. Since the model isn't uncertain about them, entropy can't surface them. The model doesn't know it doesn't know.

Random sampling avoids this trap by giving every sample equal selection probability. The 184 actual DoS examples it picked were enough to teach the model the new attack signature, boosting attack recall from 4% to 65% — a 16× improvement from labeling just 0.07% of Wednesday's traffic.

The honest reading of this result: pure entropy-based AL is the wrong tool for out-of-distribution attack detection. More appropriate alternatives include diversity-based sampling, query-by-committee, density-weighted methods that prioritize feature-space coverage, or learning-to-reject approaches that explicitly flag inputs as "outside training distribution." This is an important finding for the project, not a failure.

Limitations

Dataset. CIC-IDS2017 is known in the literature for producing high scores across most classifiers in random-split settings, with critique of duplicates and label issues. Stage 1 numbers should be read as a model-vs-model comparison on a fixed benchmark, not evidence about real-world IDS performance. Stage 2 and Stage 3 results are more realistic but still limited to two days of one dataset.

Active learning scope. This is a single-round AL implementation with one batch of 500 samples. Real systems iterate over many rounds, often with smaller batches and richer uncertainty measures (margin sampling, query-by-committee, expected model change). Treat this as a proof of concept, including its negative finding.

Hyperparameters. None of the model hyperparameters were tuned via grid search or cross-validation. Reasonable defaults were used throughout.

Neural Network Classifier for Network Intrusion Detection

Overview

This writeup covers two experimental stages using a Multilayer Perceptron (MLP) neural network on the CIC-IDS2017 intrusion detection benchmark:

- Stage 1 — Random-split baseline: standard 80/20 stratified split on the full ~2.1M-row dataset, used to establish neural network performance under favorable conditions and provide a direct comparison to the classical models.
- Stage 2 — Temporal split (Tuesday → Wednesday): train on Tuesday traffic, test on Wednesday traffic. A realistic deployment scenario that tests generalization to unseen attack types.

Stage 1: Random-Split Baseline

Approach

We trained a Multilayer Perceptron (MLP) on the same full-dataset random split used for the classical models, loading the saved train/test split (X_train.npy, X_test.npy, etc.) to ensure direct comparability. The neural network was chosen as the advanced model because deep architectures can learn non-linear,

hierarchical feature representations — an advantage for network traffic data where attack patterns involve complex combinations of flow statistics.

Methodology

Features were standardized using StandardScaler fit on the training set only to prevent data leakage. Class imbalance was handled using sample_weight to apply weights equivalent to scikit-learn's class_weight="balanced", since MLPClassifier does not natively support class weighting. Architecture: three hidden layers (128 → 64 → 32 neurons), ReLU activation, Adam optimizer, early stopping with a 10% validation fraction and patience of 5 iterations. Train set: 1,676,908 rows (1,369,772 benign + 307,136 attacks). Test set: 419,227 rows (342,443 benign + 76,784 attacks).

Results

Accuracy: 0.9932 | Attack Precision: 0.9673 | Attack Recall: 0.9964 | Attack F1: 0.9817 | Macro F1: 0.9887 | Training iterations: 13

```
PS C:\Users\macia\OneDrive\Desktop\testets> python member3_stage1.py
Stage 1: Random-split baseline (FULL DATASET, ~2.1M rows)

Loading .npy files...
  Train -> 1,676,908 rows   normal: 1,369,772   attack: 307,136
  Test  -> 419,227 rows   normal: 342,443   attack: 76,784

Training Neural Network (128-64-32, relu, adam)...
  fit: 1629.4s   iterations: 13

Neural Network (128-64-32)
  predict time: 10.6s
  accuracy: 0.9932
  classification report:
              precision    recall  f1-score   support

      normal     0.9992    0.9925    0.9958    342443
      attack     0.9673    0.9964    0.9817     76784

    accuracy                         0.9932    419227
   macro avg     0.9833    0.9944    0.9887    419227
weighted avg     0.9934    0.9932    0.9932    419227

  confusion matrix (rows=true, cols=pred):
[[339858   2585]
 [   274  76510]]

================================================================================
STAGE 1 SUMMARY -- random split 80/20 on FULL dataset
================================================================================
                           accuracy  normal_precision  normal_recall  normal_f1  ...  attack_f1  macro_f1  weighted_f1  predict_sec
model                                                                             ...
Neural Network (128-64-32)    0.9932            0.9992         0.9925     0.9958  ...     0.9817    0.9887       0.9932      10.5551

[1 rows x 10 columns]

Saved -> member3_stage1_results.csv
PS C:\Users\macia\OneDrive\Desktop\testets>
```

Discussion

Under a random split, the Neural Network achieved 0.9817 attack F1 and 0.9932 accuracy. This is competitive with the classical models on the same data: Decision Tree (F1 = 0.9985), kNN (F1 = 0.9913), and Logistic Regression (F1 = 0.9430). All four model architectures sit within roughly 5 percentage points of each other, suggesting that under standard random-split evaluation, this problem is well within reach of any reasonable model — including a linear one. The neural network missed only 274 attacks out of 76,784 in the test set, with training converging in 13 iterations before early stopping triggered.

Stage 2: Temporal Split (Tuesday → Wednesday)

Motivation

Stage 1 results reflect an idealized setting. In real deployment, models must handle traffic containing attack types they have never seen. Tuesday contains brute-force attacks (FTP-Patator, SSH-Patator); Wednesday contains DoS attacks (Hulk, GoldenEye, Slowloris, Slowhttptest) plus Heartbleed — entirely different feature signatures.

Methodology

Same architecture and preprocessing as Stage 1. The temporal-split .npy files provided by the team (X_train_tue.npy, y_train_tue.npy, X_test_wed.npy, y_test_wed.npy) were used directly, with sanity-check assertions confirming the row counts matched expected Tuesday and Wednesday sizes before training began. Training set (Tuesday): 445,645 rows — 431,813 benign + 13,832 attacks (3.1% attack rate). Test set (Wednesday): 691,406 rows — 439,683 benign + 251,723 attacks (36.4% attack rate).

Results

Accuracy: 0.6361 | Attack Precision: 0.5340 | Attack Recall: 0.0037 | Attack F1: 0.0074 | Macro F1: 0.3923 | Training iterations: 18. Of 251,723 Wednesday attacks, the neural network caught only 934 — a 0.37% recall.

```
C:\Users\Deniss Garcia\OneDrive\Escritorio\Neural Network>python member3_temporal_split.py
Setup: tuesday_wednesday

Loading .npy files...
  Train (Tuesday)  -> 445,645 rows   normal: 431,813   attack: 13,832
  Test  (Wednesday) -> 691,406 rows   normal: 439,683   attack: 251,723

  Sanity check passed: row counts match expected Tuesday/Wednesday sizes.

Training Neural Network (128-64-32, relu, adam)...
  fit: 406.2s   iterations: 18

Neural Network (128-64-32)
  predict time: 4.4s
  accuracy: 0.6361
  classification report:
              precision    recall  f1-score   support

      normal     0.6364    0.9981    0.7772    439683
      attack     0.5340    0.0037    0.0074    251723

    accuracy                         0.6361    691406
   macro avg     0.5852    0.5009    0.3923    691406
weighted avg     0.5991    0.6361    0.4969    691406

  confusion matrix (rows=true, cols=pred):
[[438868    815]
 [250789    934]]

================================================================================
TEMPORAL SPLIT SUMMARY -- setup: tuesday_wednesday
================================================================================
                            accuracy  normal_precision  normal_recall  ...  macro_f1  weighted_f1  predict_sec
model                                                                  ...
Neural Network (128-64-32)    0.6361            0.6364         0.9981  ...    0.3923       0.4969       4.3814

[1 rows x 10 columns]

Saved -> member3_temporal_results.csv
Share member3_temporal_results.csv with the team for the 4-way comparison!

C:\Users\Deniss Garcia\OneDrive\Escritorio\Neural Network>
```

Discussion

The neural network's performance collapsed under temporal distribution shift, dropping from 0.9817 attack F1 on the random split to 0.0074 on the temporal split. This places it alongside the classical models, which all collapsed to under 5% attack recall as well: Decision Tree 0.08%, kNN 0.20%, Logistic Regression 4.03%, Neural Network 0.37%. The model trained on Tuesday's brute-force patterns — many small repeated login attempts with specific TCP flag patterns — cannot recognize Wednesday's DoS

patterns, which manifest as high-volume timing-based traffic with fundamentally different feature signatures. The MLP's multi-layer architecture, despite being theoretically capable of learning more abstract feature representations than the classical models, did not provide any practical advantage here. It learned the wrong representations: those specific to brute-force traffic, which do not generalize to DoS attacks. Precision held up reasonably — when the model does flag something as an attack, it is correct about 53% of the time — but it almost never flags anything. This finding strengthens the case for the active learning experiment: if even a deep neural network fails on cross-day distribution shift, then no amount of model engineering alone solves this problem. The shift between attack types is a fundamental data property, not an architectural limitation.

Limitations

No hyperparameter tuning. Architecture and training settings were chosen based on standard practice, not grid search or cross-validation.

Training time. ~27 minutes for the 1.7M-row Stage 1 training on CPU. While the Stage 2 run was faster (~7 minutes for 445k rows), real-time retraining on live high-volume traffic would still require GPU acceleration.

Single dataset and day pair. Results reflect one specific dataset (CIC-IDS2017) and one specific temporal split (Tuesday → Wednesday). Generalization to other intrusion detection datasets or other day-pair combinations is unverified.

Active learning not applied to NN. The Stage 3 active learning experiment was performed using Logistic Regression rather than the neural network, due to retraining-cost constraints. Extending active learning to the NN would require either a faster retraining strategy or partial fine-tuning rather than full retraining each round.

Conclusion

This project set out to compare machine learning models for network intrusion detection on CIC-IDS2017. The original framing was a standard "which model works best?" comparison. What we found, however, reshaped the question.

Under standard random-split evaluation (Stage 1), all four model architectures performed similarly well, with attack F1 scores ranging from 0.9430 to 0.9985. By that measure, intrusion detection looks like a solved problem.

When we evaluated the same models on a realistic temporal split (Stage 2), every model collapsed. Attack recall dropped to under 5% across all four architectures, including the deep neural network. The Decision Tree fell from catching 99.87% of attacks to catching 0.08%. This was not a model-selection problem — it was a fundamental property of how training and test data relate to each other.

The active learning experiment (Stage 3) tested whether selective labeling could recover the lost performance. We implemented entropy-based uncertainty sampling, a textbook technique. To our surprise, it failed completely: of 500 carefully selected Wednesday samples, zero were attacks. The model was not uncertain about Wednesday's DoS attacks — it was confidently wrong about them, classifying them as benign with high probability. A random-sampling control caught 184 attacks by chance and boosted recall 16× over entropy-based selection.

Three findings emerge from this work. First, random-split evaluation dramatically overstates real-world performance — the gap between Stage 1 and Stage 2 is the difference between a working detector and a non-functional one. Second, distribution shift between attack types defeats every model architecture we tested, including deep learning. This is a data problem, not a model problem. Third, pure uncertainty-

based active learning has a critical blind spot: when models are confidently wrong about out-of-distribution inputs, entropy cannot surface those samples for labeling.

The larger lesson is that evaluation methodology matters as much as model choice in intrusion detection. Standard benchmarks measure pattern recognition on familiar attacks; real deployment requires generalization to unfamiliar ones. Bridging that gap is more an engineering and data challenge than an algorithmic one, and our results suggest that the most promising research directions lie in selective labeling strategies, out-of-distribution detection, and continually updated training data, rather than in pursuing ever-more-complex model architectures.